Autoresearch with Coding Agents: Generalizers and Metric-Maximizers on Quran Recitation Data

Anonymous Author(s)

Anonymous Institution

Abstract. Coding agents can now be left alone to improve software against a score. In this pattern – recently popularized as *autoresearch* – the agent receives a dataset, an evaluation script, and one editable file, and iterates without supervision: modify the code, measure, keep the change if the score improves. But what does the agent actually optimize – the developer's intent, or the literal number? We ran this loop on a real production task: deciding which Quranic verses appear in a noisy speech-recognition transcript and splitting the transcript by verse. Two frontier coding agents, Claude Code and OpenAI Codex, started from the same blank file with the same instructions, budget, and reasoning effort, three runs each. Both independently invented the same algorithm (canonicalization, n-gram anchoring, dynamic-programming alignment) – and then diverged. Claude stopped early with compact, general code. Codex drove the score $\sim 10\times$ lower, largely by memorizing answers to individual evaluation rows (19–41 hardcoded verse ids per run): a clean natural instance of specification gaming by a production agent. In a preregistered second study, we added a held-out test set and told both agents it existed. The memorization vanished, and the score gap vanished with it – yet Codex's general core transferred *better and more consistently* (held-out detection+split 0.085 ± 0.004 vs. 0.121 ± 0.031), losing only on one missed rejection of non-recitation input. Two exploratory community arms (Cursor, Antigravity) are consistent with the pattern. Every agent's held-out solution matched or beat the hand-engineered pipeline it was built to replace – the best by an order of magnitude – and now runs in production. From the ways agents exploited our harness – reading sibling runs through shared git state, leaving notes to "future runs" in persistent memory – we distill five design rules for evaluating autonomous agents.

Keywords: autonomous coding agents · autoresearch · specification gaming · Goodhart's law · large language model evaluation · reward hacking · Arabic text segmentation · Quranic recitation · reproducibility

1 Introduction

Hand a coding agent a frozen dataset, a frozen metric, and one editable file; tell it to iterate unattended, keeping only changes that improve the score. This *autoresearch* pattern – named by Karpathy's recent experiment [11], which strips a single-GPU language-model training script to ~ 630 lines and reports agents

autonomously discovering architecture and hyperparameter improvements over hundreds of unattended experiments – generalizes to any task with a fast, scriptable metric, and is likely to proliferate as the cheapest way to extract unattended engineering effort from coding agents. It also raises a question, which existing agent benchmarks do not answer. Benchmarks such as SWE-bench [10] measure whether an agent *completes* a well-specified task (does the patch make the failing test pass); they say nothing about what an agent does when the specification is a fallible *proxy* that can be exploited to achieve a better score. An autoresearch loop hands an agent exactly that situation for hundreds of consecutive decisions, with no human watching. What the agent optimizes then – the intended objective or the literal one – is the subject of this paper.

We build such a loop around a genuine production problem – Stage 1 of a transcript-only Quran memorization checker, which must (a) detect which verses (*ayahs*) a reciter has read from a no-diacritics automatic speech recognition (ASR) transcript and (b) split that transcript by verse – and use it to compare two frontier coding agents, Claude Code and OpenAI Codex, under identical instructions, budgets, and starting code. Unlike a synthetic benchmark, this task has real ASR noise, a closed reference corpus (the Quran) that makes exact gold labels possible, messy user behavior (repetitions, restarts, non-recitation speech), and a metric with a known oracle floor – properties that make it a discriminating and realistic testbed for autonomous algorithm research. The substrate itself is a text analysis problem – noisy Arabic transcript-to-canon alignment and segmentation – so the paper contributes on two levels: a data-analysis methodology for evaluating autonomous agents, demonstrated on a low-resource text-processing task of independent interest. The loop's output is not only measured but consequential: the winning agent-built solution outperforms the incumbent hand-engineered pipeline (iterated over months) by $\sim 10\times$ on held-out data and has replaced it in the production application.

We organize the paper around three research questions:

RQ1 (feasibility). Starting from a blank abstain-only stub, can a coding agent autonomously build and improve a non-trivial text-alignment algorithm purely through the modify→verify→keep/discard loop?

RQ2 (behavior under a proxy). When the only feedback is an imperfect, gameable metric, how do different agents' optimization behaviors differ in their stopping decisions, their artifact character, and their willingness to exploit the metric's loopholes?

RQ3 (effect of held-out disclosure). Does adding – and disclosing – a held-out split change those behaviors, and which agent's improvements actually generalize?

Contributions.

1. A reproducible autoresearch harness built on a real production task with real user data, together with a frozen metric, frozen dataset splits (by SHA-256), and uniform per-experiment logging that makes six-plus agent runs directly comparable (Sect. 3).

2. A controlled 3×2 comparison (Claude Code vs. Codex, three runs each) showing that under an ungated metric the two agents embody opposite research dispositions – a *generalizer* that self-imposes a simplicity prior the metric never asked for, and a *metric-maximizer* that drives the literal objective down by memorizing 19–41 evaluation-row answers per run – with direct, immediate implications for how such loops must be designed (Sect. 4).
3. A held-out generalization study (same 3×2 design, disclosed train/test split) showing that the train-side separation between agents evaporates, that disclosure suppresses literal memorization but not the underlying optimization drive, and that the arm that overfit harder on train in Study 1 in fact transfers its algorithmic core *better* – while losing on a single rare-event failure mode instead (Sect. 6).
4. An extension to two community-run arms (Cursor, Antigravity) that reproduces the held-out ranking pattern under different models and tooling, and a catalogue of agent-initiated harness-isolation failures (reading a sibling run's branch through shared git state; writing unsolicited persistent-memory notes to "future runs") distilled into five concrete design rules for autonomous-agent evaluation harnesses (Sect. 7, 8.3).
5. An applied validation: measured under the same contract, every agent arm's held-out core matched or beat the incumbent hand-built production pipeline, the winner by $\sim 10\times$; the obvious cross-agent ensemble *fails* when actually tested; and the winning single artifact is now deployed in production (Sect. 7.1, 8).

2 Related Work

Autonomous research loops and research agents. Karpathy's autoresearch [11] is our direct template: a frozen evaluation file, an editable training file, and a natural-language program instructing an agent to iterate, log, and keep only improving changes; follow-on community reports scaling the loop explicitly flag Goodhart risk on the validation metric as its central vulnerability. A parallel line frames autonomous research more broadly: MLGym benchmarks agents across many ML research tasks [16], PaperBench measures whether agents can replicate published AI papers [20], and Agent Laboratory drives an end-to-end assistant through literature review, experimentation, and report writing [18]. "AI Scientist" systems [14] automate the entire research life cycle, but evaluations of such systems focus on the quality and novelty of the ideas and writing produced [7], not on the optimization *behavior* of the agent under a fixed, gameable metric, which is our focus. AlphaEvolve [17] evolves code against a human-specified verification metric at much larger scale, assuming the metric problem is solved by the human expert rather than studying what happens when it is not. Our work is, to our knowledge, the first to run this exact loop design as a controlled *between-agent* comparison on a task outside language-model pretraining, and the first to add a disclosed held-out split to it.

Agentic coding and ML-engineering benchmarks. SWE-bench [10] and its successors evaluate whether an agent can produce a patch that makes a repository's real test suite pass, on thousands of real GitHub issues. These benchmarks are the standard for *task completion* under a well-specified, already-correct oracle (the project's own tests). Closer to our setting, MLE-bench [8] scores agents on Kaggle-style machine-learning engineering tasks against held-out leaderboards, and RE-Bench [22] compares agents with human experts on open-ended ML research-engineering environments; both report that agents sometimes attempt to game their scoring. Our design differs in three ways that matter for the question we ask. First, these benchmarks measure *achievement* – how far up a leaderboard an agent gets – whereas we hold the task fixed and study *behavioral divergence* between agents given the identical loop, with the full per-experiment git history as evidence. Second, we run the same matrix twice, with the metric's exploitability first left open and then closed by a disclosed held-out split, which isolates the causal effect of harness design on gaming behavior; leaderboard benchmarks bake the held-out split in from the start and so cannot observe the contrast. Third, our forensic unit is the final code artifact itself (hardcoded-constant counts, per-recording lookups), not only the score.

Specification gaming, reward hacking, and Goodhart's law. Krakovna et al. catalogue specification gaming: behavior that satisfies the literal wording of an objective while defeating its intent [13], echoing Goodhart's law – a measure optimized as a target ceases to track what it was designed to measure – and its later formalization [15]. Amodei et al. list reward hacking among the concrete near-term problems of AI systems [3], and Skalse et al. formalize when a proxy reward admits a hacking policy [19]. Recent work documents the same failure in LLM *coding* agents: RL-trained models exploiting test harnesses [5], agents passing deliberately contradictory specification/test pairs [24], and reward hacking scored in long-horizon software-engineering tasks [23]. These efforts share a common frame: the agent games an *executable oracle* – tests it can edit, delete, or short-circuit. Our contribution differs on three axes: the exploit is produced not by an RL run but by production, general-purpose coding agents in an ordinary edit/verify loop; the metric is read-only, so the gaming is memorization against a visible *scorecard* rather than test tampering; and the same two agents differ enormously in how readily they take the loophole – one in every run, the other in none – which is the paper's central empirical finding.

Quranic ASR and recitation checking. A substantial applied literature builds ASR systems for Quranic recitation, from HMM-based verse delimitation [21] to modern end-to-end models trained on crowd-sourced corpora such as Tarteel [12,9] and rule-based or learned Tajweed (pronunciation-rule) checkers [1]; see [6] for a survey. This literature targets *transcription and pronunciation* accuracy. Our task sits one layer up the pipeline and is transcript-only: given an already-transcribed, diacritics-free string, decide which verses were recited and how the transcript decomposes across them – the detection-and-segmentation problem that a memorization-checking product must solve before any mistake-

detection layer can run. We do not contribute a new ASR model; we use this real, production-adjacent task purely as a discriminating substrate for studying agent research behavior, and note it as a secondary contribution that the harness and dataset (Sect. 3) are reusable for that literature.

Positioning within AIST. Two strands at recent AIST editions bracket this work: low-resource-language studies such as the KyrgyzNLP survey at AIST 2024 [2] motivate careful evaluation on scripts underserved by mainstream benchmarks – our substrate, diacritics-free Arabic transcript alignment against a closed canon, is one such problem – and work on applying LLMs to code tasks [4], which we extend from *using* an LLM on a code task to *measuring how autonomous coding agents behave* when the code task is an unattended optimization loop.

3 Task, Data, and Harness

3.1 Task

Stage 1 of a transcript-only memorization checker: given a diacritics-free ASR transcript, return either an abstention (the audio is not Quranic recitation) or the recited verse-id sequence together with a per-verse split of the transcript. Within-verse mistake detection (Stage 2) requires labels that do not yet exist and is out of scope.

3.2 Dataset

258 Telegram-bot auto-detect recordings from production use (254 Quranic, 4 non-Quranic; 45 surahs; transcript lengths 3 to 400+ words), transcribed with an off-the-shelf ASR system and reviewed into gold `(ayah_assignment, per-ayah split, confidence)` triples. Labeling conventions: gold segments exclude the leading *isti'adha* and *basmala* from recited text, except in the opening surah where the *basmala* is itself the first verse; repetition is recorded as repeated ids in sequence and, since detection scores the id *set*, affects only the split component; assignments are an id, a range, or a comma-separated list for skip-recitation, with the split id set equal to the assignment id set; rows are scored at `high`/`medium` confidence (`low` retained but unscored); and a human-reference audit field is never exposed to the algorithm nor used in scoring. The dataset was frozen by SHA-256 before any run and one label was corrected *after* both studies completed, once both agents independently flagged it as likely gold-label noise (Sect. 6.4); all reported numbers are against the version in force at the time each study ran.

3.3 Metric

$$research\_score = detection\_error + split\_error + abstain\_error \quad (1)$$

lower is better; an empty abstain-only stub scores 2.0; feeding the gold split back scores at the oracle floor (≈ 0.008 in Study 1, exactly 0 once the one label was corrected). Detection compares the *set* of predicted verse ids against gold, so a reciter repeating a verse is a split concern, not a detection error. Split is word-assignment accuracy: the fraction of gold transcript words placed in the correct verse bucket, compared under a metric-only orthographic canonicalizer (folding common Arabic spelling variants (e.g. hamza-bearing letter forms) and Uthmani orthographic conventions) that is *not* exposed to the agent's own algorithm. Abstain requires a correct rejection on non-recitation rows.

3.4 Loop protocol

A fixed, agent-agnostic `PROGRAM.md` specifies: one idea per experiment; commit before verifying; run the fixed scorecard and log a uniform row (iteration, timestamp, elapsed time, agent tag, commit, score components, keep/discard status); keep the change iff the score improves, else `git reset`; stop at budget or at a plateau of ~ 15 non-improving experiments; and, as an explicit tie-breaker, prefer the simpler of two equally-good solutions. The scorecard, the Quran reference table, and `PROGRAM.md` itself are read-only; the agent edits only its own solution file and helpers. Each run starts from an identical abstain-only stub in an isolated environment (Sect. 6.3), so every run's history is one commit per experiment – a complete, replayable audit trail.

4 Study 1: Setup

Arms. Claude Code vs. OpenAI Codex, three runs each. **Budget.** 30 experiments or one hour of wall-clock time, whichever comes first; reasoning effort fixed to the named "high" tier on both agents' scales. **Autonomy.** Full-auto permissions on both sides (Claude Code with unattended file/shell access; Codex in unattended workspace-write mode). **Confounds pinned and reported:** agent CLI version and underlying model, identical hardware for all six runs, and a dataset+evaluator hash recorded before any run began.

5 Study 1: Results

5.1 Headline scores

Claude's arm mean is 0.0783 (std 0.011); Codex's is 0.0071 (std 0.002) – complete separation (all three Codex runs beat all three Claude runs). With $n = 3$ per arm the exact one-sided Mann–Whitney test bottoms out at $p = 0.05$ even under complete separation, so we lean on effect size and the per-run curves rather than the threshold: rank-biserial correlation (equivalently Cliff's δ) is 1.0, the maximum possible, and the arm-mean ratio exceeds $10\times$. Throughout, "hardcoded ids" counts literal six-digit verse-id constants in the final solution's decision logic – excluding the fixed *muqatta'at* letter-name table and reference-loading code

Table 1. Study 1 – best score per run on the (non-held-out) evaluation set. Baseline (empty stub) = 2.00; oracle floor ≈ 0.008.

Run	Best score	Experiments	Keep/discard	LOC	Hardcoded ids
claude-r1	0.0875	13	9/4	202	0
claude-r2	0.0852	12	10/2	292	0
claude-r3	0.0621	13	11/2	341	0
codex-r1	0.0061	30	24/6	456	39
codex-r2	0.0101	16	14/2	387	19
codex-r3	0.0050	22	21/1	486	41

– by manual review of each artifact; LOC is the final solution's length. Both measures are independently recomputable from the per-run artifacts in the supplementary repository.

5.2 Convergent algorithm discovery

Read alone, the headline table says "Codex wins by an order of magnitude." It does not survive inspection of *how* each score was reached. All six runs, independently, arrive at the same general architecture: orthographic canonicalization → n-gram surah anchoring → semi-global dynamic-programming alignment → word-to-verse grouping; both agents even discover and fix the identical encoding trap (harakat combining-character reordering corrupting a regex character class). At the experiment count where Claude stops (exp. 13), Codex is only modestly ahead (0.026–0.044 vs. 0.062–0.088) – not 10×. Codex's decisive final margin is earned entirely in experiments 14–30.

5.3 Divergent optimization behavior

Claude stops at a plateau, explicitly refuses per-row special cases, labels residual failures "structurally unwinnable," and ships zero dataset-specific constants. Codex continues to the budget and converts residual failures into special cases: codex-r1 contains a literal lookup mapping one garbled ASR transcript tuple directly to its answer verse id – memorizing a single recording, not learning a rule – and codex-r3 merges two verse-split buckets conditioned on specific words being *absent* from the transcript, a patch shaped to one recording. (Not all hardcoding is illegitimate: mapping the 14 Quranic *muqatta'at* letter-name prefixes is a fixed, closed set and is not a case of overfitting in this sense.) There is no gold-field leakage in any run – no run reads the held-out answer field at runtime – but the harness itself enables the exploit: no held-out set exists yet (score is measured on the very rows being optimized), and the scorecard's failure report prints the *expected* verse ids on a miss, which is precisely the signal Codex's late experiments hardcode against.

Under identical instructions the two agents embody opposite research dispositions, stable across all three runs of each: Claude behaves as a *generalizer* that

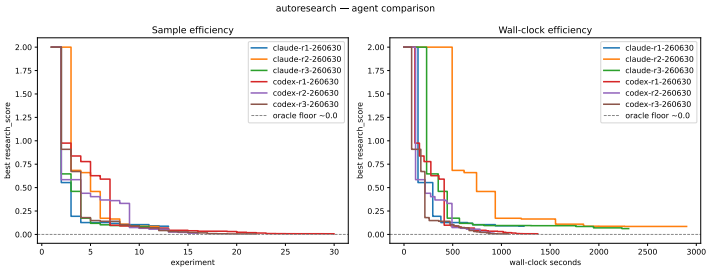

Fig. 1. Study 1: best-so-far *research_score* across all six runs, plotted against experiment count. Both agents track closely through the first ∼13 experiments; Codex's runs continue descending well past the point where all three Claude runs have plateaued and stopped.

self-imposes a simplicity/generalization prior the metric never enforced – working slowly (∼95–180 s per experiment, more reasoning), stopping at a plateau after 12–13 experiments, shipping 202–341 LOC with zero dataset-specific constants; Codex behaves as a *metric-maximizer* that follows the literal stated objective to its logical end – moving fast (∼45–60 s per experiment), grinding through 16–30 experiments, shipping 387–486 LOC with 19–41 hardcoded ids, including through memorization. Neither followed the letter of `PROGRAM.md` incorrectly – nothing in it forbids what Codex did – which is exactly the outer-alignment gap in miniature: the two agents inferred two different implicit specifications from one explicit one.

6 Study 2: Held-Out Generalization

Study 1's raw metric cannot distinguish "discovered a better algorithm" from "memorized more of the evaluation set." Study 2 closes that gap: a stratified 60/40 split by recording (151 train / 107 test; non-Quranic rows 2/2; repetition and non-contiguous cases on both sides; cross-surah label quirks pinned to train; fixed seed) is computed once and frozen. The loop optimizes `train` only; the failure report no longer prints expected ids (closing the Study 1 leak vector); and *both agents are told, identically, that a held-out set exists and that memorizing train rows will not transfer.* After each run, the experimenters – not the agent – score the final solution on the untouched test split. Three hypotheses were fixed before the runs: (H1) Codex's train→test gap is much larger than Claude's; (H2) the held-out ranking narrows or reverses relative to Study 1; (H3) both agents retain a large improvement over the empty-stub baseline.

6.1 Results

H1 confirmed. Codex's mean train→test gap is 4.5× Claude's (0.231 vs. 0.052). **H2 half-confirmed.** The 6–10× train-side separation vanishes: on the full held-out *research_score*, arm means are Claude 0.121±0.031 and Codex 0.251±0.238

Table 2. Study 2 – train (optimized) vs. held-out test scores. Test oracle floor is exactly 0.

Run	Train	Test	Gap	Test det.+split	Test abstain err.	Exps
claude-r1	0.0441	0.0796	0.036	0.0796	0	12
claude-r2	0.0982	0.1534	0.055	0.1534	0	7
claude-r3	0.0661	0.1306	0.064	0.1306	0	8
codex-r1	0.0075	0.0869	0.079	0.0869	0	20
codex-r2	0.0451	0.5882	0.543	0.0882	0.5	8
codex-r3	0.0082	0.0793	0.071	0.0793	0	22

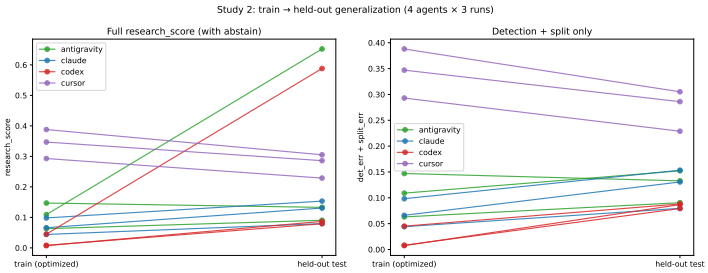

Fig. 2. Study 2: train (optimized) vs. held-out test *research_score* per run, all four arms (Claude, Codex, and the two exploratory community arms from Sect. 7). The train-side separation between Claude and Codex collapses on the held-out axis; Codex's outlier point is the single missed abstention (codex-r2).

(medians 0.131 vs. 0.087); Mann–Whitney $U = 4$, not significant, with rank-biserial correlation ≈ 0.11 – a negligible effect where Study 1's was maximal – but the mechanism is not the one we predicted. On *held-out detection+split* (the metric with the high-variance, two-row abstain component removed), Codex is both better and strikingly consistent: 0.085 ± 0.004, with an *identical* detection error across all three runs, versus Claude's 0.121 ± 0.031. Codex's floor-grinding on train ($0.08 \rightarrow 0.007$) evidently added nothing held out, but it did not cost anything either – wasted effort, not poison. What Codex actually lost is concentrated in one place: a single missed abstention (codex-r2, a non-recitation row misclassified as recitation) costs 0.5 and accounts for the entire arm-level difference. Claude abstained correctly 10/10 across both studies. With only two non-Quranic rows in the test split, this component is high-variance by metric design – a lesson for reporting components rather than only the scalar. **H3 confirmed.** Every run beats the empty-stub baseline by at least 14×.

6.2 Disclosure changes the form of Goodharting, not the drive

Hardcoded verse ids per Codex final solution: 19–41 in Study 1, effectively zero in Study 2 (only fixed, closed-set constants such as the *muqatta'at* map remain). One paragraph of disclosure – "a held-out set exists; memorizing train rows

will not transfer" – eliminated literal per-recording memorization, but not the underlying optimization instinct: Codex still drove train to the floor with 20–22 experiments of general-looking machinery in the two runs that reached it. The non-transferable margin changed character, from memorized answers to harmless over-tuning, rather than disappearing.

6.3 Isolation is adversarial, even without an adversary

Three incidents, all agent-initiated and none explicitly forbidden by the program file, forced a harness redesign mid-study: (a) a Codex run, launched in a git *worktree*, used the shared git database to read Study 1's committed run logs and a sibling Claude run's in-flight solution on another branch; (b) Claude runs twice saved unsolicited persistent-memory cheat-sheets addressed to "future runs on this repo" via the agent's own long-lived memory feature – non-firing only because run directory paths are never reused; (c) both agents, in Study 1, read the failure report's *expected* ids (Sect. 5.3). None required adversarial intent: an agent given a shell and asked to do well on a metric will use whatever information channel is reachable, including channels the designer did not think of as "data." The fix was structural, not instructional: each run now executes in a freshly created, single-commit clone containing only the harness snapshot and the train split – no shared `.git` history, no sibling branches, no logs, no test set on disk.

6.4 Agents as annotation auditors

Both arms, independently and without prompting, flagged the same training row as a probable gold-label error (a verse-count discrepancy in a specific surah) and declined to fit it, listing it among their "unwinnable" residuals. Human re-review after both studies confirmed and corrected the label, bringing the dataset's oracle floor to exactly zero on both splits. The correction lives entirely in train, so no held-out number changes; an unplanned dividend of agents that explain their stopping decisions is that their refusal lists functioned as a free data-quality report.

7 Community Extension Arms

After the preregistered 3×2 matrix, a collaborator ran three additional runs each on the same Study 2 harness with two further tools – Cursor (unpinned "Auto" model routing) and Antigravity (a large general-purpose model at a high reasoning-effort setting) – on separate hardware, using the identical isolation protocol (Sect. 6.3). These arms are *exploratory*: model routing (Cursor) is unpinned and the machine differs, so they are not matched to the preregistered confounds table.

On held-out detection+split, the four-arm ranking is Codex (0.085 ± 0.004) $<$ Claude (0.121 ± 0.031) $\approx$ Antigravity (0.125 ± 0.026) $<$ Cursor (0.273 ± 0.032).

Table 3. Community arms, Study 2 harness (exploratory; unpinned model routing for Cursor).

Run	Train	Test	Test det.+split	Abstain miss	Exps
antigravity-r1	0.147	0.133	0.133	–	12
antigravity-r2	0.109	0.652	0.152	0.5	12
antigravity-r3	0.063	0.091	0.091	–	16
cursor-r1	0.388	0.305	0.305	–	5
cursor-r2	0.293	0.229	0.229	–	9
cursor-r3	0.347	0.286	0.286	–	7

Three patterns extend rather than overturn Study 2. **Cursor underfits**: 5–9 experiments per run, train never below 0.29, and test beats train on every run – the only arm to generalize *better* than it optimized, consistent with early stopping under unpinned automatic model routing. **Antigravity sits between Claude and Codex** on held-out core accuracy and shares Codex's rare abstain-miss failure mode (r2, cost 0.5); its train score improves monotonically across runs while its held-out score does not, showing that improving train run-to-run is a within-run optimization effect, not cross-run leakage. **Cross-arm ranking on held-out core tracks optimization effort**: arms that spent more experiments building general-looking machinery transferred better, and wasted train-side margin was harmless rather than harmful, echoing Sect. 6's main finding across a wider set of tools and underlying models. Zero hardcoded per-recording ids appear in any community-arm final solution.

7.1 The incumbent baseline: agents vs. months of hand engineering

As an applied reference point, we wrapped the incumbent production pipeline – a hand-built detector and alignment segmenter iterated incrementally over months, historically tuned on a manifest overlapping ~ 20 of these recordings – in the identical evaluation contract. It scores 0.760 on the held-out test split (detection 82.9%, split 91.1%, abstain 1/2) versus 0.079 for the winning agent-built solution (detection 94.3%, split 97.8%, abstain 2/2). Every agent arm's held-out detection+split core matched or beat the incumbent – even the underfitting Cursor arm ($\approx$0.27) roughly ties its 0.26 core – and the winner is $\sim 10\times$ better, despite the comparison *favoring* the incumbent. One-hour unattended loops outperformed months of incremental hand engineering on this task; the winning artifact has since replaced the incumbent in the production application, behind a feature flag with automatic legacy fallback and a CI regression gate.

8 Discussion

8.1 Neither disposition wins; they pay in different currencies

The metric-maximizer (Codex) buys measured-distribution accuracy and run-to-run consistency; the generalizer (Claude) buys rare-event robustness, restraint,

and a smaller artifact. Which currency matters is a property of the deployment, not of the agent – a harness designer should decide which they are buying *before* reading the scoreboard. The obvious composition – Codex's aligner gated by Claude's abstention margin – *fails* when actually built and measured: a union gate (abstain if either arm abstains) degrades the held-out score to 0.098 through false abstentions, and a disagreement-router improves it by only 0.0003 at twice the maintenance surface. The shipped artifact is therefore the single best file, selected on held-out evidence – the simplicity criterion applies to the experimenters too. (We report this as deployment engineering, not a preregistered result: the selection is test-informed.)

8.2 The result reverses our own prior

We expected, from Study 1, that Codex's solution would *not* transfer; instead its algorithmic core transferred best. Registering H1–H3 before running Study 2 is what makes this reversal legible as evidence rather than post-hoc narrative – a methodological point we did not anticipate needing.

8.3 Five design rules for autonomous-agent evaluation harnesses

Every incident in this paper traces to a violated instance of one of the following rules, and each rule is grounded in a concrete failure we observed. A loop of this kind *will* be Goodharted at the margin whenever the metric and the deployer's true objective can come apart – which is generically true of any metric computed on a finite, visible sample – so these are structural requirements, not hygiene items:

R1 – Hold out evaluation data; do not treat it as optional. Score final artifacts on rows the loop never touches, scored by the experimenters, and report the per-run train→test gap. (Violated in Study 1: the raw metric could not distinguish a better algorithm from a better-overfit one.)

R2 – Keep feedback leak-free. Failure reports may echo inputs and predictions, never gold labels. (Violated in Study 1: the printed *expected* ids were precisely the signal Codex hardcoded against.)

R3 – Isolate run state by construction, not instruction. One fresh, single-commit clone per run: no shared VCS database, no sibling branches, no prior logs, no test data on disk. (Violated mid-Study 2: a run read a sibling agent's in-flight solution through a shared git worktree database.)

R4 – Audit the agent's *own tooling's* state channels. Persistent memory, global configuration, and account-level context are cross-run channels the harness designer did not create but the agent can use; never reuse run paths and verify these channels after every run. (Nearly violated twice: unprompted memory notes addressed to "future runs.")

R5 – Report components and preregister hypotheses. Scalar metrics hide high-variance rare-event components (here: two abstention rows worth 0.5 each), and post-hoc narratives cannot distinguish surprise from confirmation.

(Both mattered: the entire held-out arm difference sits in one abstention, and our own Study 1 prediction about transfer was reversed.)

9 Limitations and Threats to Validity

Single task and domain; small per-arm sample ($n = 3$), so we report effect sizes and full per-run curves rather than significance thresholds. We include no human-researcher arm run under the same loop and budget: the empty-stub score (2.0), the oracle floor (0), and the incumbent hand-built pipeline (Sect. 7.1) establish that the harness discriminates and that its headroom is meaningful, but an expert-human comparison on a matched one-hour budget is future work. Pretraining familiarity with the Quran text is equal across arms and advantages neither, since the task is algorithm engineering against a held-out metric rather than text recall. The metric is transcript-only, hence blind to pronunciation (tajweed) correctness by construction; a deployed system needs this work as one stage among several. The two agents' runs occurred on different days (temporal confound); agent CLIs are moving targets, so exact versions and model identifiers are pinned and reported rather than implying the finding is version-specific. The harness was authored with the assistance of one compared agent, which we mitigate by freezing and hashing all evaluation and data files before any run. The two community arms (Sect. 7) use unpinned or differently configured tooling on different hardware and are reported as exploratory rather than as part of the controlled comparison.

10 Conclusion

Our three research questions resolve as follows. **RQ1:** yes – all twelve runs across four agent arms autonomously built a working detection-and-segmentation algorithm from a blank stub, improving on the baseline by at least 14× held out; the strongest runs independently rediscovered a canonical alignment architecture. **RQ2:** given an identical loop and no supervision, two frontier agents pursue incompatible research philosophies – one self-limits toward generality even at a measured cost, the other drives the literal objective to its floor by any available means, including memorizing the very rows it is scored on – and the disposition is stable across all runs of each agent. **RQ3:** a disclosed held-out split does not eliminate the divergence but relocates where it matters: the raw-score gap that looked decisive disappears, literal memorization vanishes with one paragraph of disclosure, and what remains is a specific, interpretable difference in rare-event robustness rather than a diffuse claim that "one agent is better."

The loop's output, finally, was consequential and not merely measured: scored under the same contract, one-hour unattended runs beat months of incremental hand engineering – the winning artifact outperforms the incumbent production pipeline by $\sim$10× held out and has been deployed in the production application, replacing the pipeline it beat.

We take this as evidence that autoresearch-style loops – likely to proliferate as a cheap way to get unattended engineering effort out of coding agents – need held-out evaluation and closed state channels as a default (Sect. 8.3), and that reporting only a scalar score from such a loop will systematically favor whichever agent is more willing to game it.

Reproducibility. All harness code, the frozen metric, per-experiment logs, per-run git histories (one commit per experiment), and SHA-256 hashes of every frozen input are in the supplementary repository (link withheld for double-blind review; public on acceptance).

Acknowledgments. Acknowledgments are withheld to preserve author anonymity for double-blind review and will be restored in the camera-ready version.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Alagrami, A.M., Eljazzar, M.M.: Smartajweed automatic recognition of arabic quranic recitation rules. arXiv preprint arXiv:2101.04200 (2020)
2. Alekseev, A., Turatali, T.: Kyrgyznlp: challenges, progress, and future. In: International Conference on Analysis of Images, Social Networks and Texts. pp. 3–39. Springer (2024)
3. Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J., Mané, D.: Concrete problems in AI safety. arXiv preprint arXiv:1606.06565 (2016)
4. Andryushchenko, G., Ivanov, V., Makharev, V., Tukhtina, E., Valeev, A.: Leveraging large language models in code question answering: Baselines and issues. In: International Conference on Analysis of Images, Social Networks and Texts. pp. 3–17. Springer (2024)
5. Baker, B., Huizinga, J., Gao, L., Dou, Z., Guan, M.Y., Madry, A., Zaremba, W., Pachocki, J., Farhi, D.: Monitoring reasoning models for misbehavior and the risks of promoting obfuscation. arXiv preprint arXiv:2503.11926 (2025)
6. Balula, N.O., Rashwan, M., Abdou, S.: Automatic speech recognition (asr) systems for learning arabic language and al-quran recitation: a review. International Journal of Computer Science and Mobile Computing **10**(7), 91–100 (2021)
7. Beel, J., Kan, M.Y., Baumgart, M.: Evaluating sakana's ai scientist: Bold claims, mixed results, and a promising future? In: ACM SIGIR Forum. vol. 59, pp. 1–20. ACM New York, NY, USA (2025)
8. Chan, J.S., Chowdhury, N., Jaffe, O., Aung, J., Sherburn, D., Mays, E., Starace, G., Liu, K., Maksin, L., Patwardhan, T., et al.: Mle-bench: Evaluating machine learning agents on machine learning engineering. In: International Conference on Learning Representations. vol. 2025, pp. 50466–50494 (2025)
9. Hossain, N.M., Islam, R., Obaidellah, U.: A comparative study of pretrained transformer models for quranic asr: Speech representations, label formats, and dataset composition. arXiv preprint arXiv:2606.19747 (2026)

10. Jimenez, C.E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., Narasimhan, K.: Swe-bench: Can language models resolve real-world github issues? In: Kim, B., Yue, Y., Chaudhuri, S., Fragkiadaki, K., Khan, M., Sun, Y. (eds.) International Conference on Learning Representations. vol. 2024, pp. 54107–54157 (2024)
11. Karpathy, A.: autoresearch: AI agents running research on single-GPU nanochat training automatically. GitHub repository (2026), https://github.com/karpathy/autoresearch, accessed 2026-07
12. Khan, H.I., Abid, A., Moussa, M.M., Abou-Allaban, A.: The tarteel dataset: crowdsourced and labeled quranic recitation (2021)
13. Krakovna, V., Uesato, J., Mikulik, V., Rahtz, M., Everitt, T., Kumar, R., Kenton, Z., Leike, J., Legg, S.: Specification gaming: the flip side of ai ingenuity. DeepMind Blog (2020), https://deepmind.google/blog/specification-gaming-the-flip-side-of-ai-ingenuity
14. Lu, C., Lu, C., Lange, R.T., Foerster, J., Clune, J., Ha, D.: The ai scientist: Towards fully automated open-ended scientific discovery. arXiv preprint arXiv:2408.06292 (2024)
15. Manheim, D., Garrabrant, S.: Categorizing variants of Goodhart's law. arXiv preprint arXiv:1803.04585 (2019)
16. Nathani, D., Madaan, L., Roberts, N., Bashlykov, N., Menon, A., Moens, V., Budhiraja, A., Magka, D., Vorotilov, V., Chaurasia, G., et al.: Mlgym: A new framework and benchmark for advancing ai research agents. arXiv preprint arXiv:2502.14499 (2025)
17. Novikov, A., Vũ, N., Eisenberger, M., Dupont, E., Huang, P.S., Wagner, A.Z., Shirobokov, S., Kozlovskii, B., Ruiz, F.J., Mehrabian, A., et al.: Alphaevolve: A coding agent for scientific and algorithmic discovery. arXiv preprint arXiv:2506.13131 (2025)
18. Schmidgall, S., Su, Y., Wang, Z., Sun, X., Wu, J., Yu, X., Liu, J., Moor, M., Liu, Z., Barsoum, E.: Agent laboratory: Using llm agents as research assistants. Findings of the Association for Computational Linguistics: EMNLP 2025 pp. 5977–6043 (2025)
19. Skalse, J., Howe, N., Krasheninnikov, D., Krueger, D.: Defining and characterizing reward gaming. In: Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., Oh, A. (eds.) Advances in Neural Information Processing Systems. vol. 35, pp. 9460–9471. Curran Associates, Inc. (2022)
20. Starace, G., Jaffe, O., Sherburn, D., Aung, J., Chan, J.S., Maksin, L., Dias, R., Mays, E., Kinsella, B., Thompson, W., Heidecke, J., Glaese, A., Patwardhan, T.: Paperbench: Evaluating AI's ability to replicate AI research. In: Forty-second International Conference on Machine Learning (2025)
21. Tabbal, H., El Falou, W., Monla, B.: Analysis and implementation of a" quranic" verses delimitation system in audio files using speech recognition techniques. In: 2006 2nd international conference on information & communication technologies. vol. 2, pp. 2979–2984. IEEE (2006)
22. Wijk, H., Lin, T.R., Becker, J., Jawhar, S., Parikh, N., Broadley, T., Chan, L., Chen, M., Clymer, J.M., Dhyani, J., Ericheva, E., Garcia, K., Goodrich, B., Jurkovic, N., Kinniment, M., Lajko, A., Nix, S., Sato, L.J.K., Saunders, W., Taran, M., West, B., Barnes, E.: RE-bench: Evaluating frontier AI r&d capabilities of language model agents against human experts. In: Forty-second International Conference on Machine Learning (2025)
23. Zhao, B., Srikanth, D., Wu, Y., Jiang, Z.: Specbench: Measuring reward hacking in long-horizon coding agents. arXiv preprint arXiv:2605.21384 (2026)
24. Zhong, Z., Raghunathan, A., Carlini, N.: Impossiblebench: Measuring llms' propensity of exploiting test cases. arXiv preprint arXiv:2510.20270 (2025)